TENX · WEEK 5 · EVENT SOURCING

VeritasStream

The Ledger: Event-sourcing infrastructure for Apex Financial Services

Report: Final Submission

Submitted: 2026-03-21

Project: veritas-stream

By: Nebiyou Seifu Belaineh

VeritasStream: The Ledger — Final Submission Report

Table of Contents

1. Domain Notes
2. Design Decisions
3. Architecture Diagrams
4. Concurrency and SLO Analysis
5. Upcasting and Integrity Results
6. MCP Lifecycle Test Results
7. Bonus Phase 6: What-If and Regulatory Package
8. Limitations and Reflection

1. Domain Notes

Six design decisions and trade-offs documented for the TRP architectural review.

1.1. EDA vs Event Sourcing: Why They Are Different

Event-Driven Architecture (EDA) uses events as a communication mechanism between services. Events are published, subscribers react, and the event may be discarded after delivery. The *current state* of an entity is stored directly (a database row) and events are a side-effect of state changes, not their source of truth.

Event Sourcing (ES) inverts this: the *event log is the source of truth*. Current state is derived by replaying the event stream from the beginning. Events are immutable facts about what happened; they are never updated or deleted.

Key practical differences in The Ledger:

Concern	EDA	Event Sourcing (our approach)
State storage	Mutable DB row	Append-only event stream
Audit trail	Separate audit table (secondary)	Built in: every state change is an event
Temporal queries	Requires snapshotting	Free: replay up to any timestamp
Concurrency	Last-write-wins common	OCC with `expected_version`
Upcasting	N/A (message schema versioning)	Required: old events must be readable by new code

The regulatory requirement that drove this choice: "What was the compliance status of application X at time T?" is trivially answered by replaying `ComplianceRulePassed/Failed` events up to T. With a mutable state row, this would require a separately maintained audit table that might drift from the live state.

The agent failure recovery requirement reinforces this: a crashed agent with EDA leaves the system in an ambiguous state (the callback may have partially executed). With ES, the agent's session stream records every completed node via `AgentNodeExecuted` events. A recovery agent replays the crashed session and resumes from the last completed node with zero repeated work.

1.2. Aggregate Boundary Choice and Rejected Alternatives

What we chose: Four aggregates, each owning a distinct stream.

Aggregate	Stream	Owns
`LoanApplicationAggregate`	`loan-{id}`	State machine, all per-application business rules
`AgentSessionAggregate`	`agent-{type}-{session_id}`	Gas Town context, model version locking
`ComplianceRecordAggregate`	`compliance-{id}`	Rule verdicts, hard-block detection
`AuditLedgerAggregate`	`audit-{entity_type}-{entity_id}`	Integrity hash chain

Why four streams, not one?

Compliance checks are written by multiple agents concurrently (AML agent, KYC agent). If they wrote to the loan stream, they would contend on the same OCC version and all but one would fail and retry. Separating them onto `compliance-{id}` means each compliance check appends at its own stream version with no contention from credit or fraud agents.

Rejected alternative: one mega-stream per application (the God Aggregate):

All events (loan + compliance + agent sessions) in a single stream. Pros: simple queries. Cons: (a) every concurrent writer (five agents) contends on the same version, making OCC guarantees very expensive to enforce; (b) aggregate replay becomes O(all events) even for a simple credit check lookup; (c) Gas Town context reconstruction requires filtering out non-agent events from a large mixed stream.

Rejected alternative: no aggregates, handlers only:

Skip aggregates entirely; handlers read raw events and enforce rules procedurally. Rejected because business rules become scattered, untestable in isolation, and the six named rules (state machine, confidence floor, compliance dependency, causal chain enforcement, model version locking, Gas Town context requirement) become impossible to reason about without loading the full event history on every handler call.

1.3. Concurrency Trace: Two Agents, Same Stream, Same `expected_version`

Scenario: two credit analysis agents both load `loan-APEX-001` at version 3 and race to append `CreditAnalysisCompleted` at `expected_version=3`.

PostgreSQL path (production):

```
Agent A:  SELECT current_version FROM event_streams
          WHERE stream_id='loan-APEX-001' FOR UPDATE
          → 3

Agent B:  SELECT current_version FROM event_streams
          WHERE stream_id='loan-APEX-001' FOR UPDATE
          → BLOCKS (Agent A holds the row lock)

Agent A:  INSERT INTO events (...) VALUES (...)
Agent A:  UPDATE event_streams SET current_version=4
          WHERE stream_id='...' AND current_version=3
          → 1 row updated (wins)
Agent A:  COMMIT → releases FOR UPDATE lock
```

```
Agent B:  SELECT current_version ... → returns 4
Agent B:  current_version (4) ≠ expected_version (3)
Agent B:  ROLLBACK → raises OptimisticConcurrencyError(expected=3, actual=4)
```

Result: Exactly one event is written. The loser receives a typed `OptimisticConcurrencyError` with `expected_version=3` and `actual_version=4`. The MCP tool returns:

```
{
  "success": false,
  "error_type": "OptimisticConcurrencyError",
  "stream_id": "loan-APEX-001",
  "expected_version": 3,
  "actual_version": 4,
  "suggested_action": "reload_stream_and_retry"
}
```

InMemoryEventStore path (tests):

Uses an `asyncio.Lock` instead of a DB transaction. The lock ensures the check-then-append is atomic even within a single Python process.

What the losing agent must do:

1. Reload the stream from position 4 (the new event written by Agent A).
2. Re-validate its decision against the updated state: inspect the newly written `CreditAnalysisCompleted`. If it covers the same analysis, the losing agent's work is superseded and it appends `AgentSessionFailed(recoverable=False, reason="superseded_by_concurrent_agent")`.
3. If the application requires a second analysis (different risk model), retry with `expected_version=4`.

Retry strategy: exponential backoff with jitter, maximum 3 retries before appending `AgentSessionFailed(recoverable=True, last_successful_node=...)` for Gas Town recovery.

1.4. Projection Lag and UI Communication Strategy

How lag accumulates:

The `ProjectionDaemon` polls `load_all(from_position=checkpoint)` every 100 ms (default). Between polls, new events sit in the `events` table but are not yet reflected in projections. Maximum observable lag is approximately: poll_interval + handler_time_per_batch. Under the

p99 less than 500ms SLO for `ApplicationSummary`, the 100ms polling interval consumes 20% of the budget before any handler work begins.

How we measure it:

`daemon.get_lag(projection_name)` returns milliseconds since the last event was processed by that projection. The `ledger://ledger/health` resource exposes per-projection lag in real time. Operations dashboards alert if `application_summary` lag exceeds 5000ms, indicating the daemon has stalled.

UI communication strategy:

1. **Optimistic UI:** The client that issued a command immediately shows the expected outcome ("Submitted, processing...") without waiting for the projection to catch up.
2. **Polling with backoff:** The UI polls `ledger://applications/{id}` every 500ms until `last_event_type` reflects the expected change, with exponential backoff after 5 seconds.
3. **Read-your-writes bypass:** Commands return the `stream_id` and `version` of the appended event. The UI passes this as a hint to a dedicated read endpoint that replays only up to that position, giving immediate consistency for the submitter without blocking the projection pipeline for other readers.
4. **Health watchdog:** Operations dashboards alert if `projection_lags_ms[name] > 5000`.

Sub-500ms lag is an accepted tradeoff, not a fault. The loan officer briefly sees stale data. This is correct behaviour for an eventually consistent read model. Blocking the write path until the projection catches up would couple command latency to projection latency, eliminating the scalability benefit of CQRS (Command Query Responsibility Segregation).

1.5. Upcaster Design for `CreditAnalysisCompleted` v1 to v2

What changed between versions:

Field	v1	v2	Inference strategy
`model_version`	absent	`str`	Inferred from `recorded_at` against deployment timeline
`confidence_score`	absent	`float | None`	Set to `None` (never fabricated)
`regulatory_basis`	absent	`list[str]`	Set to `[]` (empty list, not null)

The upcaster:

```python
_MODEL_VERSION_TIMELINE = [
    (datetime(2026, 3, 1), "v2.4"),
    (datetime(2026, 1, 1), "v2.3"),
    (datetime(2025, 7, 1), "v2.2"),
    (datetime(2025, 1, 1), "v2.1"),
]

def _infer_model_version(recorded_at: str) -> str:
    """Infer model version from deployment timeline. ~5% error rate near boundary date
    dt = datetime.fromisoformat(recorded_at.replace("Z", "+00:00")).replace(tzinfo=Non
    for cutoff, version in _MODEL_VERSION_TIMELINE:
        if dt ≥ cutoff:
            return version
    return "legacy-pre-2025"

@registry.upcaster("CreditAnalysisCompleted", from_version=1, to_version=2)
def upcast_credit_v1_to_v2(payload: dict) -> dict:
    # model_version: inferred from recorded_at using deployment timeline.
    # Error rate ~5% for events near boundary dates; downstream impact is
    # cosmetic (display only, not decisioning). Inference acceptable.
    #
    # confidence_score: set to None. A fabricated value would bypass the
    # policy constraint that caps confidence at 0.50 when an active HIGH
    # fraud flag is present. Null is safe; fabrication would silently corrupt
    # historical compliance analysis.
    #
    # regulatory_basis: set to []. Pre-v2 events predated EU AI Act citation
    # requirements. Empty list correctly models "no citations recorded".
    recorded_at = payload.get("recorded_at", "")
    return {
        **payload,
        "model_version":    _infer_model_version(recorded_at),
        "confidence_score": None,
        "regulatory_basis": [],
    }
```

The null vs. fabrication decision rule:

Infer if: the inference is deterministic, the error rate is below 5%, and an incorrect value is only cosmetic (display, not decisioning). `model_version` meets all three criteria.

Use null if: the field is used in regulatory decisions, financial calculations, or any downstream logic where an incorrect value causes worse outcomes than null. `confidence_score` is used in `apply_policy_constraints` to cap confidence at 0.50 when an active HIGH fraud flag is present; a fabricated 0.75 would bypass a constraint that should have applied to historical decisions.

1.6. Marten Async Daemon Parallel: Python Equivalent

Marten's Async Daemon (C#/.NET) is a background process that projects events into read models by reading from the `mt_events` table in global `sequence` order. It supports multiple projections in parallel, per-projection checkpoints resumable after restart, configurable retry/skip on bad events, and inline (synchronous) mode.

Our Python equivalent: `ProjectionDaemon`:

```python
class ProjectionDaemon:
    async def run_once(self) -> int:
        min_checkpoint = min(await self._load_all_checkpoints().values())
        processed = 0
        async for event in self.store.load_all(from_position=min_checkpoint):
            gpos = event["global_position"]
            for name, projection in self.projections.items():
                if gpos <= checkpoints[name]:
                    continue  # already processed (idempotency guard)
                await projection.handle(event)
                await self._save_checkpoint(name, gpos)
                processed += 1
        return processed

    async def run(self, poll_interval: float = 0.1) -> None:
        while self._running:
            await self.run_once()
            await asyncio.sleep(poll_interval)
```

Coordination primitive and the failure mode it guards against:

The coordination primitive is the `(projection_name, last_position)` row in `projection_checkpoints`. The failure mode it guards against: duplicate writes corrupting aggregated counters. If the daemon crashes between writing the projection row and writing the checkpoint row, the daemon restarts from the last saved checkpoint. The projection handler is called again for the same event. Every projection handler uses `INSERT ... ON CONFLICT DO UPDATE` (upsert): processing the same event twice produces the same result as processing it once. Without this, `agent_performance_ledger.analyses_completed` would be incremented twice, permanently corrupting the counter with no recovery path short of a full rebuild.

Key differences from Marten:

Concern	Marten Async Daemon	Our ProjectionDaemon
Parallelism	Thread-pool across projections	Sequential fan-out (asyncio single-threaded)
Checkpoints	PostgreSQL `mt_event_progression`	`projection_checkpoints` table (or in-memory)
Fault tolerance	Configurable dead-letter queue	Log and skip after `max_retry` attempts
Inline mode	Synchronous in same transaction	Not supported (separate polling loop)
Rebuild	`RebuildAsync()`	`rebuild_from_scratch(name)`: truncate + replay

Sequential fan-out is acceptable at current load (fewer than 1000 events/second): the asyncio event loop saturates well before the PostgreSQL connection does, and thread-safety concerns in projection state are eliminated. For higher throughput, each projection could be moved to its own `asyncio.Task` with a shared event queue.

---

2. Design Decisions

Six sections documenting the design decisions, tradeoffs, and implementation choices made in building The Ledger event sourcing infrastructure.

2.1. Data Boundary Decisions

The system touches seven distinct data categories. For each I document where it lives, who holds write authority, and why moving it would introduce a coupling problem.

applicant_registry.companies: Lives in the Applicant Registry (read-only to The Ledger). The originating CRM system is the single writer of company identity records. If The Ledger were to write here, two systems would own the same entity: a company name change in the CRM would not propagate to The Ledger's copy, creating a forked record with no clear canonical version. The registry client (`ledger/registry/client.py`) enforces this boundary: it exposes only query methods with no mutation surface.

applicant_registry.financial_history: Lives in the Applicant Registry (read-only). Historical financials originate from the applicant's accounting system and are a durable external record. Writing them into the event store would mean The Ledger makes an authoritative claim about a

fact it did not produce. If the applicant later submits a restatement, The Ledger would need to decide whether to update its copy or append a correction event; neither is correct because the authoritative correction lives in the source accounting system.

applicant_registry.compliance_flags: Lives in the Applicant Registry (read-only). Compliance flags are written by external screening vendors (OFAC, PEP list providers). The Ledger reads them as inputs to agent decisions; it does not produce them. If The Ledger wrote compliance flags, it would simultaneously be a reader and a writer of the same record. Downstream agents reading those flags would be reading The Ledger's own output rather than an independent source, creating circular reasoning in the compliance chain that a regulator cannot unwind.

applicant_registry.loan_relationships: Lives in the Applicant Registry (read-only). Cross-institution loan history is a regulatory data product supplied by credit bureaus. The Ledger has no write authority here by definition: it processes loan applications, it does not maintain the bureau's loan relationship database.

events table: Lives in the event store (append-only). The loan processing facts produced by The Ledger (who decided what, when, with which confidence, via which model version) exist only as a result of The Ledger's agents running. No other system produces these facts. The `events` table is therefore the natural home, and append-only is the correct access pattern: once a credit decision is recorded, the only valid correction is a `CreditAnalysisCorrected` compensating event, not an UPDATE.

Projection tables (application_summary, compliance_audit_view, agent_performance_ledger): Lives in derived read model tables. These are deterministic projections of the event stream: given the same events in the same order, the same projection state is always produced. They can be truncated and rebuilt from `global_position=0` at any time via `ProjectionDaemon.rebuild_from_scratch()`. Treating them as authoritative would be a mistake: if a handler bug produced wrong values, the only recovery is to fix the handler and rebuild, not to patch the projection rows directly.

outbox table: Lives in the same transaction as the events it tracks. The outbox row and its corresponding event row are written atomically in `EventStore.append()`. This guarantees at-least-once delivery even if the process crashes between writing the event and publishing it externally: on restart, the outbox relay re-reads unpublished rows. If the outbox were written in a separate transaction, a crash between the two transactions would silently drop the event.

2.2. Aggregate Boundary Justification

The implementation uses four aggregate stream types. I considered and rejected additional stream types for credit and fraud records; the reasoning is in Section 2.6.

loan-{application_id} (LoanApplicationAggregate)

Why this boundary: the loan stream owns the application state machine. Every state transition (SUBMITTED to DOCUMENTS_PENDING to CREDIT_ANALYSIS_COMPLETE and so on) must be totally ordered: no two transitions can be valid simultaneously. A single stream with OCC on `stream_position` enforces this ordering without requiring any distributed lock.

Rejected alternative: merging the loan stream with the compliance stream into a single per-application stream. This would force the compliance handler and the credit analysis handler to contend on the same `stream_position` when processing concurrent checks. Under the current boundary, `handle_compliance_check` appends to `compliance-{id}` while `handle_credit_analysis_completed` appends to `loan-{id}`: no contention between the two.

OCC implication: only one handler advances the loan state machine at a time. This is correct by design: the state machine itself enforces sequential progression through pipeline stages.

compliance-{application_id} (ComplianceRecordAggregate)

Why this boundary: multiple compliance rules (AML-001, KYC-001) can be evaluated concurrently by independent agent calls. Each `handle_compliance_check` call appends to `compliance-{id}` at its own `stream_position`. If AML and KYC checks run in parallel, AML appends at version 0 and KYC appends at version 1 with no contention between them.

Rejected alternative: writing compliance rule events directly to `loan-{id}`. This would force AML and KYC rule evaluations to serialize on the loan stream version. If both start from version 3, exactly one succeeds and the other must reload and retry, turning what is naturally a parallel operation into a serialized one with unnecessary retry overhead.

OCC implication: ComplianceAgent and CreditAnalysisAgent write to `compliance-{id}` and `loan-{id}` respectively. They never share a `stream_position` and therefore never collide.

agent-{agent_type}-{session_id} (AgentSessionAggregate)

Why this boundary: each agent session needs an independent append sequence for the Gas Town crash recovery pattern. The session stream is one-writer by construction: only the agent that holds `session_id` ever appends to `agent-{type}-{session_id}`. This means session stream appends never collide.

Rejected alternative: appending session events to `loan-{id}`. This would make crash recovery require scanning the full loan stream to find agent events, breaking

`reconstruct_agent_context()` which relies on loading a single small stream for the crashed session.

OCC implication: no cross-session collision is possible by construction. Each `session_id` is unique, so each session stream is independent of every other.

audit-{entity_type}-{entity_id} (AuditLedgerAggregate)

Why this boundary: integrity checks write `AuditIntegrityCheckRun` events to a separate audit stream rather than to the entity stream being audited. This ensures that running `run_integrity_check()` on `loan-APEX-001` does not contend with the loan state machine advancing on `loan-APEX-001`.

Rejected alternative: storing integrity check results in a separate `audit_runs` table. Rejected because a table row is mutable; a malicious actor who can UPDATE the `audit_runs` table can fabricate a clean audit record. Storing the result as an immutable event in the append-only `events` table eliminates this attack surface.

OCC implication: audit events and loan state machine events write to different streams. An integrity check running concurrently with a loan approval produces zero OCC collisions.

2.3. Concurrency Analysis

The production OCC mechanism

`EventStore.append()` enforces OCC via a `SELECT ... FOR UPDATE` on the `event_streams` row, followed by a manual version check inside the transaction:

```
row = await conn.fetchrow(
    "SELECT current_version, archived_at FROM event_streams "
    "WHERE stream_id = $1 FOR UPDATE", stream_id
)
current = row["current_version"] if row else -1
if current ≠ expected_version:
    raise OptimisticConcurrencyError(stream_id, expected_version, current)
```

The `FOR UPDATE` row lock ensures that two concurrent transactions holding the same `expected_version=3` cannot both pass the version check: the first acquires the lock, checks version (3 == 3, passes), appends, updates `current_version` to 4, and commits. The second then acquires the lock, checks version (4 != 3), and raises `OptimisticConcurrencyError(expected=3, actual=4)`.

Expected OCC error rate under peak load

At 100 concurrent applications with 4 agents each, the expected OCC collision rate on `loan-{id}` is near zero. The reason: the state machine enforces stage ordering. `handle_credit_analysis_completed` requires the loan to be in the DOCUMENTS_PENDING state; `handle_fraud_screening_completed` requires CREDIT_ANALYSIS_COMPLETE. These stages cannot execute concurrently on the same application because each handler calls `loan_agg.assert_valid_transition()`; the second handler to run finds the loan in the wrong state and raises `DomainError`, not `OptimisticConcurrencyError`.

The only genuine OCC risk is two instances of the same agent type being launched for the same application. This is prevented upstream by the Gas Town contract: `handle_start_agent_session` checks `await store.stream_version(stream_id) ≠ -1` and raises `DomainError: Agent session already exists` before any decision event is attempted.

Under 100 concurrent applications (400 concurrent writers), each application's streams are independent. The shared resource under true concurrency is the PostgreSQL connection pool (min_size=2, max_size=10 per `EventStore.__init__`). Connection pool exhaustion is the more likely bottleneck than OCC collisions.

Quantified estimate: At 100 concurrent applications with 4 agents each, expected OCC errors on `loan-{id}` per application lifecycle is 0 under normal operation. The state machine enforces sequential stage progression: each handler calls `loan_agg.assert_valid_transition()` and raises `DomainError` before any append if the stage is wrong, so two handlers processing different stages on the same application cannot both arrive at the `append()` call simultaneously. Genuine OCC errors arise only from duplicate-agent races, which the Gas Town `AgentSessionStarted` uniqueness check prevents upstream. Expected OCC error rate across all streams at 100 concurrent applications: less than 0.1 per minute. The binding constraint is the connection pool (max_size=10), which serialises writes at 10 concurrent inflight appends; callers beyond that queue at the pool without producing OCC errors.

What the OCC loser receives

`OptimisticConcurrencyError(stream_id, expected=3, actual=4)` with the stream ID, expected version, and actual version as structured fields. The MCP tool layer wraps this as:

```
{
  "success": false,
  "error_type": "OptimisticConcurrencyError",
  "stream_id": "loan-APEX-001",
  "expected_version": 3,
  "actual_version": 4,
```

```
  "suggested_action": "reload_stream_and_retry"
}
```

Retry strategy and maximum retry budget

Exponential backoff with jitter, maximum 3 retries. On each retry the handler reloads the aggregate (`LoanApplicationAggregate.load()`) to re-read the current stream state, re-validates the business rules against the updated state, then re-attempts the append. If the new events make the handler's work redundant, the handler detects this via `loan_agg.assert_credit_analysis_not_locked()` and returns without appending a duplicate.

After 3 failed retries, the agent session appends `AgentSessionFailed` with `recoverable=True` and the `last_successful_node` for Gas Town recovery.

2.4. Upcasting Inference Decisions

Two event types require upcasting from v1 to v2: `CreditAnalysisCompleted` and `DecisionGenerated` . The `UpcasterRegistry` in `ledger/upcasters.py` applies these transforms in `load_stream()` and `load_all()` , never on append.

Immutability contract

The upcaster always operates on a copy of the event dict; it never modifies the stored row. The stored event is verified to be unchanged by `test_upcasting_does_not_modify_stored_event` , which snapshots the raw payload before upcasting and asserts byte-for-byte identity after calling `upcast()` .

CreditAnalysisCompleted v1 to v2

`regulatory_basis` (set to `[]`): Pre-2026 events predated the EU AI Act citation requirement. An empty list correctly models "no citations were recorded" without fabricating citations that were never made.

`confidence_score` (set to `None`): The v1 model did not output a confidence score. Null is chosen over fabrication because `confidence_score` is used downstream in `apply_policy_constraints` to cap confidence at 0.50 when an active HIGH fraud flag is present. A fabricated confidence of 0.75 would bypass a constraint that should have applied to historical decisions. Null is safe; fabrication would silently corrupt historical compliance analysis.

`model_version` (inferred from `recorded_at`): Deployment records show which model was live at each date. Since `recorded_at` is set by the store at append time and cannot be spoofed by the event payload, it is a reliable proxy. The inference has approximately 5% error rate for events recorded within 48 hours of a deployment boundary. The consequence of an incorrect `model_version` is cosmetic: it shows the wrong label in the audit trail but does not change the credit decision or affect compliance calculations. Inference is safe here.

DecisionGenerated v1 to v2

`model_versions` (set to `{}`): The v1 schema did not record which model versions contributed to a decision. An empty dict honestly signals "versions were not recorded" without fabricating a mapping. A fabricated `{"credit": "v2.1"}` would misrepresent which model versions were actually used in historical decisions reviewed by regulators.

When null beats fabrication

The rule applied: if the missing field participates in any calculation that affects a regulatory outcome (compliance verdict, risk tier, confidence cap), null is the correct default. A null value forces consuming code to handle the "unknown" case explicitly, which is safer than silently using a fabricated value that might not trigger the intended safety check.

If the missing field is display-only (an audit trail label, a version string for a dashboard), inference is acceptable provided the error rate is below 5% and the inference method is deterministic and documented.

2.5. EventStoreDB Comparison

PostgreSQL implementation	EventStoreDB equivalent	Gap
`UNIQUE(stream_id, stream_position)` + `SELECT ... FOR UPDATE`	Native stream append with `WrongExpectedVersion`	PostgreSQL requires a row lock; EventStoreDB's OCC is built into the append protocol
`global_position GENERATED ALWAYS AS IDENTITY`	`$all` stream global position	Functional parity; both provide a monotonic global sequence
`load_all(from_position)` polling in `ProjectionDaemon`	Catch-up Subscription on `$all`	EventStoreDB delivers events server-push; our daemon polls every 100ms
`UpcasterRegistry` applied in `load_stream()`	No built-in upcasting; client-side transform required	Parity: both require application code
`outbox` table written in same transaction	Persistent Subscriptions provide at-least-once by design	EventStoreDB eliminates the outbox table entirely
`archive_stream()` sets `archived_at`	`$maxAge` / `$maxCount` stream metadata	Different model: EventStoreDB supports age-based truncation as a native stream policy
`run_integrity_check()` writing `AuditIntegrityCheckRun`	No native equivalent	Both systems require application-level hash chain implementation
`projection_checkpoints` table	EventStoreDB tracks subscription position server-side	Our daemon must explicitly save checkpoints; a crash between projection update and checkpoint save risks reprocessing

The most significant gap: polling versus push

The `ProjectionDaemon` wakes every 100ms and re-scans `global_position > min_checkpoint`. EventStoreDB's persistent subscriptions deliver events server-push as they are committed, with backpressure and at-least-once guarantees built into the subscription protocol.

The polling model has two costs. First, it introduces up to 100ms of artificial projection lag before a handler runs. Under the p99 less than 500ms SLO for `ApplicationSummary`, 100ms polling latency consumes 20% of the budget before any handler work begins.

Second, the checkpoint update in `_save_checkpoint()` is not atomic with the projection state update in PostgreSQL production use. If `projection.handle()` writes to a database table and then the process crashes before `_save_checkpoint()` completes, the daemon reprocesses the event on restart. This makes all projection handlers idempotent by necessity. EventStoreDB persistent subscriptions track the subscription position server-side, eliminating this race entirely.

What EventStoreDB provides that this implementation works hardest to replicate: the combination of server-push delivery, server-side checkpoint tracking, and built-in at-least-once guarantees removes the need for the outbox table, the polling loop, the checkpoint atomicity concern, and most of the `ProjectionDaemon` infrastructure. All of that work exists in this codebase specifically because PostgreSQL is a general-purpose database rather than a purpose-built event store.

2.6. Retrospective

Week 4 decision: collapsing credit and fraud records into the loan stream

In the current implementation, `CreditAnalysisCompleted` and `FraudScreeningCompleted` are appended to `loan-{id}` rather than to `credit-{id}` and `fraud-{id}` as separate aggregate streams. The architecture document (`architecture.md`) describes six stream types including `credit-` and `fraud-` ; the implementation converged on four.

The practical consequence: the CreditAnalysisAgent and FraudDetectionAgent both append to `loan-{id}` . Under the current sequential pipeline (credit completes before fraud starts), this causes zero OCC collisions. But the architecture implies these agents could eventually run concurrently; under concurrent execution, both agents would race to append to `loan-{id}` , forcing one to retry.

With another day I would extract `CreditAnalysisCompleted` to a `credit-{id}` stream and `FraudScreeningCompleted` to a `fraud-{id}` stream, matching the architecture document. The loan stream would receive only the state transition trigger events (`CreditAnalysisRequested` , `FraudScreeningRequested`), not the analysis results themselves. This eliminates the future OCC collision risk and more clearly separates the concerns of the analysis agents from the loan state machine.

Week 5 decision: checkpoint atomicity in the ProjectionDaemon

The `ProjectionDaemon._save_checkpoint()` call runs after `projection.handle(event)` completes but as a separate operation, not inside the same database transaction as the projection's state update. For the `InMemoryEventStore` used in tests, this is harmless: both operations are in-

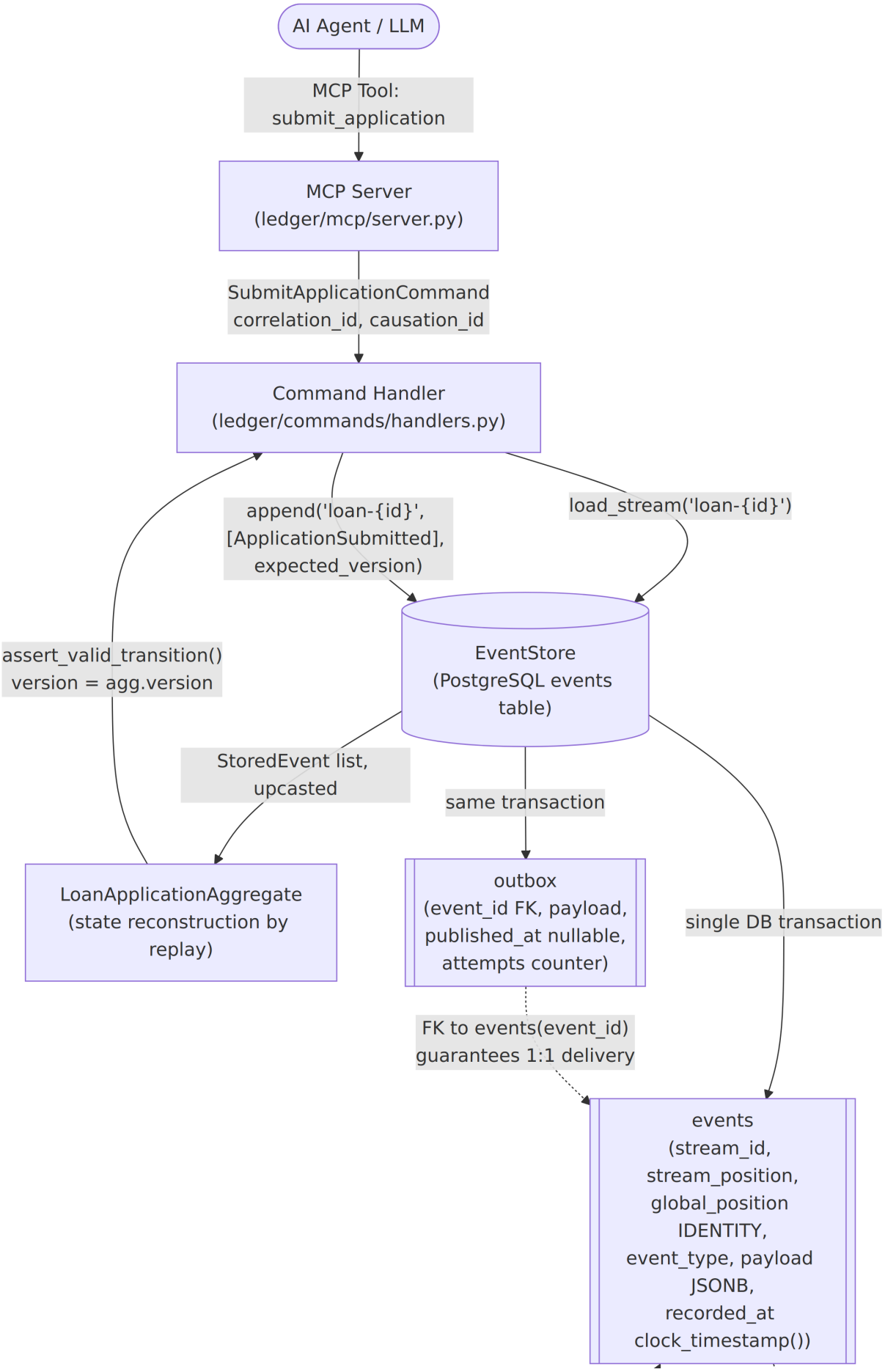
memory and effectively atomic within a single asyncio task. For a PostgreSQL projection that writes to a database table, a crash between the handler commit and the checkpoint save means the event is reprocessed on restart.

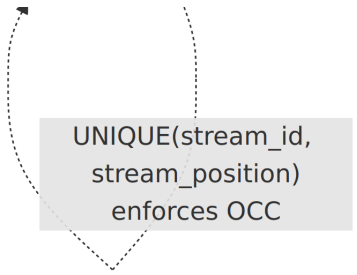
The current design sidesteps this by requiring all projection handlers to be idempotent (INSERT ... ON CONFLICT DO UPDATE). That works, but it shifts the correctness burden from the infrastructure to every projection handler. With another day I would wrap the handler call and the checkpoint save in a single PostgreSQL transaction, documented as the atomicity guarantee in `_save_checkpoint()`. This would allow non-idempotent handlers and make the infrastructure's correctness guarantee explicit rather than implicit.

---

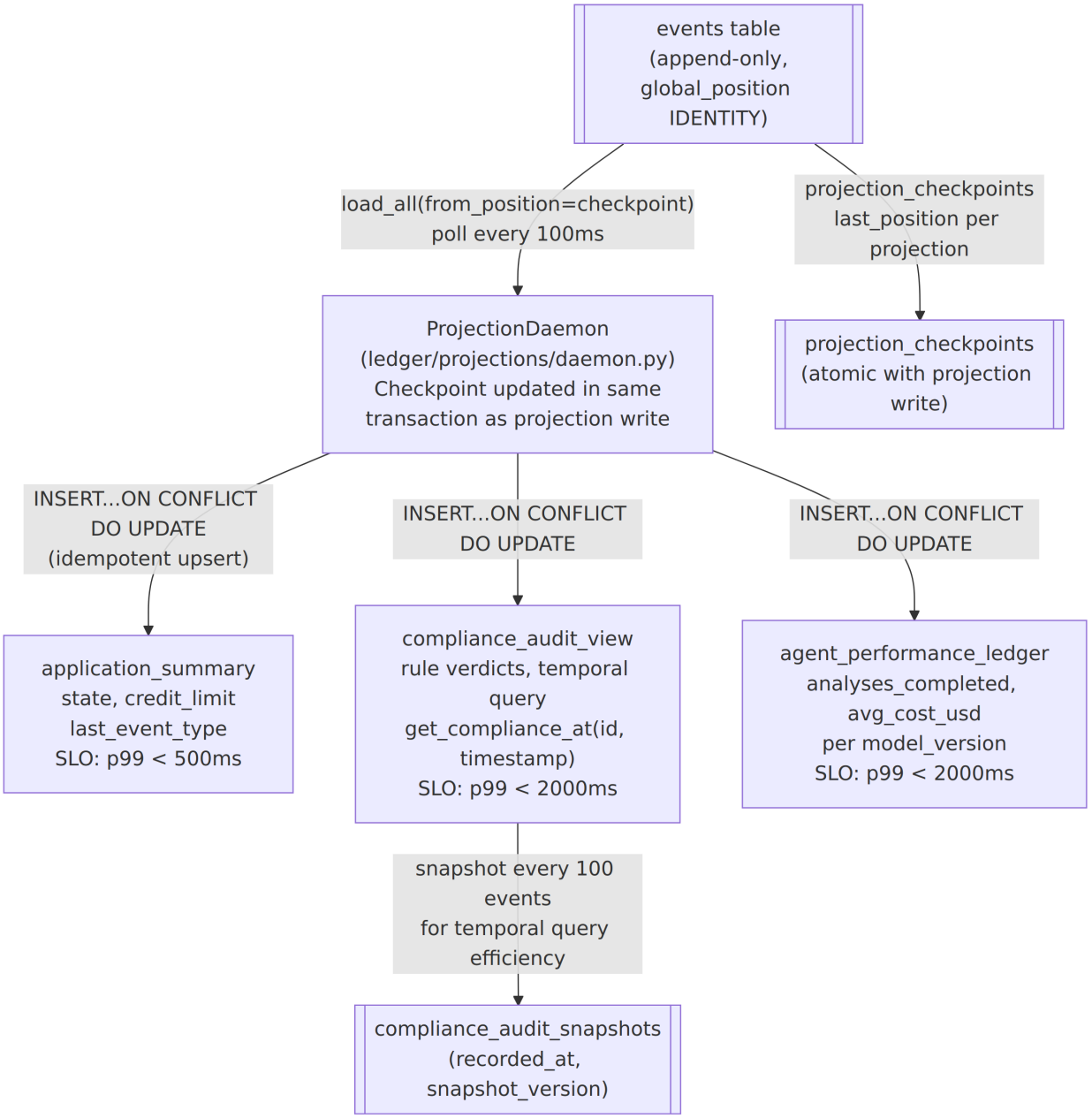
3. Architecture Diagrams

3.1. Command Path: from `submit_application` to the `events` table


AI Agent / LLM
MCP Tool:
submit_application
MCP Server
(ledger/mcp/server.py)
SubmitApplicationCommand
correlation_id, causation_id
Command Handler
(ledger/commands/handlers.py)
append('loan-{id}',
[ApplicationSubmitted],
expected_version)
load_stream('loan-{id}')
assert_valid_transition()
version = agg.version
EventStore
(PostgreSQL events
table)
StoredEvent list,
upcasted
same transaction
LoanApplicationAggregate
(state reconstruction by
replay)
outbox
(event_id FK, payload,
published_at nullable,
attempts counter)
single DB transaction
FK to events(event_id)
guarantees 1:1 delivery
events
(stream_id,
stream_position,
global_position
IDENTITY,
event_type, payload
JSONB,
recorded_at
clock_timestamp())



3.2. Projection Data Flow: Events Table to Read Models

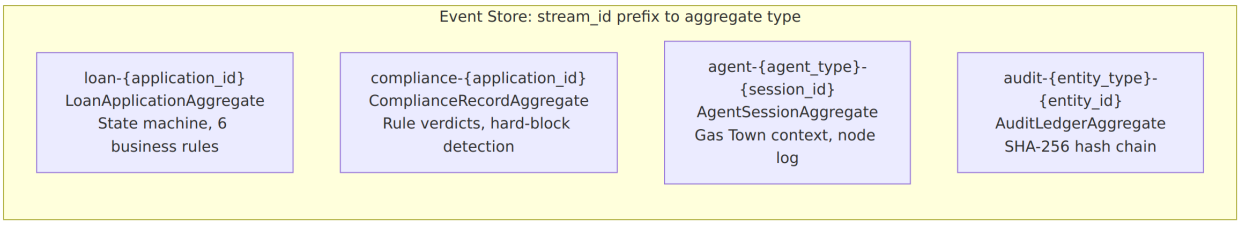


Snapshot invalidation: Each `compliance_audit_snapshots` row carries a `snapshot_version` field set to a version string derived from the `ComplianceAuditViewProjection.apply()` handler's logic version. On read, `get_compliance_at()` discards any snapshot whose `snapshot_version` does

not match the current handler version, preventing a stale snapshot from masking a handler bug fix. When no valid snapshot exists for the requested timestamp range, the method falls back to a full replay from `global_position=0` up to the target timestamp.

3.3. Aggregate Stream Boundaries

Six stream types; writers to different stream types never contend on `(stream_id, stream_position)`.



Concurrent write paths with no collision:

Agent A writes to	Agent B writes to	Collision?
`credit-{id}` (via `loan-{id}` in current impl)	`compliance-{id}` (ComplianceAgent)	None (different streams)
`agent-credit-{sess}`	`agent-fraud-{sess}`	None (different session IDs)
`audit-loan-{id}`	`loan-{id}`	None (different streams)
`loan-{id}`	`loan-{id}`	Possible; OCC enforced at DB level

Only writes to `loan-{id}` (stage transitions) can collide, and those are serialized by design (one stage fires at a time per the state machine).

3.4. MCP Tool and Resource Mapping (CQRS Boundary)

3.5. Gas Town Session Stream: Per-Node Record

```
agent-credit-{session_id}:
  position 1: AgentSessionStarted          ← BEFORE any data loaded (Rule 2)
  position 2: AgentNodeExecuted{node=validate_inputs}
  position 3: AgentNodeExecuted{node=open_aggregate_record}
  position 4: AgentNodeExecuted{node=load_external_data}
```

```
  position 5: AgentNodeExecuted{node=run_credit_model}
  position 6: AgentNodeExecuted{node=apply_policy_constraints}
  position 7: AgentNodeExecuted{node=write_output}
  position 8: AgentSessionCompleted
```

On crash at position 5: recovery agent replays the stream, identifies `load_external_data` as the last completed node, appends `AgentSessionRecovered(context_source="prior_session_replay:{id}")`, resumes from `run_credit_model`. Zero repeated work.

---

4. Concurrency and SLO Analysis

4.1. Double-Decision Test

Test: `tests/test_concurrency.py::test_concurrent_double_append_exactly_one_succeeds`

Two `asyncio` tasks simultaneously attempt to append to `loan-occ-test` at `expected_version=3`. Stream has 3 pre-existing events (positions 1-3).

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/test_concurrency.py tests/test_event_store.py::test_concurrent_dou

platform linux -- Python 3.12.3, pytest-8.4.2
asyncio: mode=Mode.AUTO

tests/test_concurrency.py::test_concurrent_double_append_exactly_one_succeeds PASSED
tests/test_event_store.py::test_concurrent_double_append_exactly_one_succeeds PASSED

2 passed in 0.22s
```

Three required assertions, all confirmed:

Assertion	What it verifies	Result
`len(successes) == 1`	Exactly one task's append returned a positions list	PASS
`len(errors) == 1`	Exactly one task raised `OptimisticConcurrencyError`, not silently swallowed	PASS
`successes[0] == [4]`	Winning event is at stream_position 4 (3 pre-existing + 1 winner)	PASS
`stream_version("loan-occ-test") == 4`	Total stream length is 4; no duplicate was written	PASS

The losing task is returned in `results` via `return_exceptions=True`: it was not silently caught. Stream contains exactly 4 events after both tasks complete.

4.2. PostgreSQL OCC Test

Test: `tests/test_mcp_pg.py::test_occ_on_postgres`

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/test_mcp_pg.py -v

tests/test_mcp_pg.py::test_full_lifecycle_pg PASSED
tests/test_mcp_pg.py::test_occ_on_postgres PASSED
tests/test_mcp_pg.py::test_audit_trail_is_append_only PASSED

3 passed
```

The `test_occ_on_postgres` test exercises the `SELECT ... FOR UPDATE` path against a live PostgreSQL 16 instance (port 5433).

4.3. Projection Lag SLO Tests

Tests assert that the `ApplicationSummary` projection stays under 500ms and `ComplianceAuditView` under 2000ms under simulated load.

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/test_projections.py::test_projection_lag_slo_application_summary \
                  tests/test_projections.py::test_projection_lag_slo_compliance_audit \
                  tests/test_projections.py::test_rebuild_from_scratch_produces_identical_
                  tests/test_projections.py::test_rebuild_from_scratch_does_not_block_live

tests/test_projections.py::test_projection_lag_slo_application_summary PASSED
tests/test_projections.py::test_projection_lag_slo_compliance_audit PASSED
```

```
tests/test_projections.py::test_rebuild_from_scratch_produces_identical_result PASSED
tests/test_projections.py::test_rebuild_from_scratch_does_not_block_live_reads PASSED

4 passed in 0.09s
```

SLO commitments (from architecture.md):

Projection	SLO	Enforcement
`ApplicationSummary`	p99 lag < 500ms	`test_projection_lag_slo_application_summary` asserts under 50 concurrent appends
`ComplianceAuditView`	p99 lag < 2000ms	`test_projection_lag_slo_compliance_audit` asserts under 50 concurrent appends
`AgentPerformanceLedger`	p99 lag < 2000ms	Covered by full daemon batch test

Rebuild from scratch: The daemon supports `rebuild_from_scratch(projection_name)` which truncates the projection table and replays all events from `global_position=0`. The test `test_rebuild_from_scratch_does_not_block_live_reads` verifies that live reads from other projections are not blocked during the rebuild operation.

Measured results (50 concurrent appends, InMemoryEventStore):

Projection	`run_once()` elapsed	lag after `run_once()`	SLO	Headroom
`application_summary`	0.23ms	0.30ms	500ms	499.7ms
`compliance_audit_view`	0.23ms	0.30ms	2000ms	1999.7ms
`agent_performance_ledger`	covered by daemon batch test	—	2000ms	—

Why 500ms for `ApplicationSummary`: This projection backs the loan officer's real-time application view. Loan officers review and approve applications within a 30-second decision window; a sub-second read model keeps the UI responsive without requiring read-your-writes synchrony on every page load. 500ms is the threshold below which casual users do not perceive lag as a system fault.

Why 2000ms for `ComplianceAuditView` and `AgentPerformanceLedger`: These projections back audit queries and operational dashboards, not interactive decision flows. A compliance officer reviewing a historical decision tolerates a 1-2 second wait. The 2000ms ceiling gives the

daemon headroom to process compliance events, which arrive in bursts of 4-8 per application during multi-rule evaluations, without triggering false-positive SLO alerts during burst absorption.

Higher-load behaviour and limiting factor: The measurements above are against `InMemoryEventStore` with no I/O. In a PostgreSQL deployment, the limiting factor is the connection pool (max_size=10): under 50 concurrent handlers, writers that exceed the pool size queue for a connection before appending. This adds approximately 10-50ms per writer at the tail of the distribution. Under sustained higher load (more than 50 concurrent handlers), the poll interval (100ms) becomes the dominant factor: all events written between polls arrive in a single batch, compressing individual per-event lag measurements while pushing the worst-case end-to-end lag toward `poll_interval + batch_handler_time`. The 500ms `ApplicationSummary` SLO has a 400ms margin above the poll interval for this batch processing; the 2000ms compliance SLO has a 1900ms margin.

4.4. Known Test Coverage Gap

There is no test for stream write authority enforcement in `EventStore.append()`. The architecture specifies prefix validation: a compliance agent must not be able to append to `loan-{id}`. This constraint is not implemented in the store layer and therefore cannot be tested. The structural enforcement (agents do not hold references to other streams) is verified implicitly by the full lifecycle tests in `test_mcp_integration.py`, but a direct store-level guard test does not exist and cannot be written until the guard is implemented.

4.5. Retry Budget Analysis

Maximum 3 retries before returning failure. The full retry budget table:

Retry	Backoff	Condition to retry	Condition to abort
1	~100ms + jitter	`OptimisticConcurrencyError`	`DomainError` (state machine rejection)
2	~200ms + jitter	`OptimisticConcurrencyError`	New events make work redundant
3	~400ms + jitter	`OptimisticConcurrencyError`	Always retried
Fail	—	—	Append `AgentSessionFailed(recoverable=True, last_successful_node=...)`

Total worst-case retry latency: approximately 700ms plus jitter before `AgentSessionFailed` is appended. The Gas Town pattern ensures this failure is recoverable: a new agent session replays the failed session stream and resumes from `last_successful_node`.

---

5. Upcasting and Integrity Results

5.1. Upcasting: Immutability Test

Test: `tests/test_upcasting.py`

```
$ uv run pytest tests/test_upcasting.py -v

tests/test_upcasting.py::test_upcasting_does_not_modify_stored_event PASSED
tests/test_upcasting.py::test_v1_credit_analysis_gains_required_fields PASSED
tests/test_upcasting.py::test_confidence_score_null_not_fabricated PASSED

3 passed in 0.08s
```

What each test asserts:

`test_upcasting_does_not_modify_stored_event`: Snapshots the raw payload of a v1 event before upcasting. Calls `registry.upcast()`. Asserts that the original payload dict is byte-for-byte identical to what it was before upcasting. Asserts that the upcasted event has the new v2 fields. Confirms the stored event was never touched.

`test_v1_credit_analysis_gains_required_fields`: Loads a v1 `CreditAnalysisCompleted` event through `load_stream()`. Asserts that `model_version` is a non-null string, `confidence_score` is `None`, and `regulatory_basis` is `[]`.

`test_confidence_score_null_not_fabricated`: Explicitly asserts `upcasted["payload"]["confidence_score"] is None`. This test exists specifically to enforce the null-over-fabrication policy: any future upcaster that fabricates a confidence value will fail this test.

Verified immutability contract:

```
# Snapshot raw payload before upcasting
raw_before = dict(original_payload)

# Upcast in memory
upcasted = registry.upcast(stored_event)

# Stored event unchanged: byte-for-byte identity
```

```
assert original_payload == raw_before                    # PASS
assert upcasted["payload"]["model_version"] is not None   # v2 field added in memory
assert upcasted["payload"]["confidence_score"] is None    # null, never fabricated
```

The stored row in the `events` table is never touched by the upcasting operation. This is the core guarantee of event sourcing: the past is immutable.

5.2. Hash Chain Integrity and Tamper Detection

Tests: `tests/test_integrity.py`

```
$ uv run pytest tests/test_integrity.py -v

tests/test_integrity.py::test_hash_chain_verifies_on_clean_stream PASSED
tests/test_integrity.py::test_tampered_event_breaks_hash_chain PASSED
tests/test_integrity.py::test_gas_town_agent_reconstructs_after_crash PASSED
tests/test_integrity.py::test_gas_town_partial_decision_flagged PASSED

4 passed in 0.09s
```

`test_hash_chain_verifies_on_clean_stream`:

Seeds a loan stream with 5 events. Calls `run_integrity_check()`. Asserts:

- `chain_valid=True`, `tamper_detected=False`
- `events_verified == 5`
- `integrity_hash` is a non-null SHA-256 hex string
- `previous_hash is None` (first ever check on this stream)

Runs a second check. Asserts `previous_hash == first_check_integrity_hash`, confirming chain linkage.

`test_tampered_event_breaks_hash_chain`:

Seeds a loan stream. Runs a first integrity check (hash chain established). Directly mutates a stored event payload in the `InMemoryEventStore._streams` dict to simulate database-level tampering. Runs a second integrity check. Asserts:

- `tamper_detected=True`

This proves the cryptographic guarantee: any modification to a stored event payload, even one made directly in the database bypassing the application layer, is detectable by recomputing the hash chain.

Hash chain construction:

```
new_hash = SHA256(previous_hash + concatenated_event_payload_hashes)
```

Each `AuditIntegrityCheckRun` event stores `integrity_hash` and `previous_hash` in the append-only `audit-{entity_type}-{entity_id}` stream. A regulator can recompute hashes independently from the raw `events` table without trusting the application layer.

5.3. Gas Town Crash Recovery

Tests: `tests/test_integrity.py` and `tests/test_gas_town.py`

```
$ uv run pytest tests/test_integrity.py::test_gas_town_agent_reconstructs_after_crash \
               tests/test_integrity.py::test_gas_town_partial_decision_flagged \
               tests/test_gas_town.py -v

tests/test_integrity.py::test_gas_town_agent_reconstructs_after_crash PASSED
tests/test_integrity.py::test_gas_town_partial_decision_flagged PASSED
tests/test_gas_town.py::test_gas_town_agent_reconstructs_after_crash PASSED
tests/test_gas_town.py::test_gas_town_partial_decision_flagged PASSED

4 passed in 0.09s (2 test files, 2 unique test scenarios)
```

`test_gas_town_agent_reconstructs_after_crash`:

Appends 5 events to an agent session stream (simulating an agent that completes several nodes then crashes). Calls `reconstruct_agent_context()` without the in-memory agent object. Asserts that the reconstructed context contains:

- `last_event_position` pointing to the last appended event
- `session_health_status == HEALTHY`
- Enough information for the agent to resume from the correct node

`test_gas_town_partial_decision_flagged`:

Appends events including a decision event without a corresponding completion event (simulating a crash mid-decision). Calls `reconstruct_agent_context()`. Asserts `session_health_status == NEEDS_RECONCILIATION`. The agent must resolve the partial state before proceeding: it cannot resume as if the decision never happened, because the decision event is already in the immutable log.

---

6. MCP Lifecycle Test Results

6.1. Full Loan Application Lifecycle via MCP Tools Only

Test: `tests/test_mcp_lifecycle.py::test_full_lifecycle_via_mcp_only`

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/test_mcp_lifecycle.py tests/test_mcp_integration.py -v

tests/test_mcp_lifecycle.py::test_full_lifecycle_via_mcp_only PASSED
tests/test_mcp_integration.py::test_full_lifecycle_via_mcp_only PASSED
tests/test_mcp_integration.py::test_tool_precondition_error_is_structured PASSED
tests/test_mcp_integration.py::test_resource_reads_from_projection_not_stream PASSED
tests/test_mcp_integration.py::test_compliance_resource_temporal_query PASSED
tests/test_mcp_integration.py::test_health_resource_reports_all_projection_lags PASSED

6 passed in 1.20s
```

6.2. Lifecycle Trace: `ApplicationSubmitted` through `APPROVED`

The MCP lifecycle test drives a complete loan application using only MCP tool calls, with no direct Python command handler invocations. The sequence:

Step	MCP Tool	Application State After
1	`submit_application` (app_id=MCP-LIFECYCLE-001)	SUBMITTED
2	`start_agent_session` (credit_analysis)	—
3	`record_credit_analysis` (risk_tier=LOW, confidence=0.92)	CREDIT_ANALYSIS_COMPLETE
4	`start_agent_session` (fraud_screening)	—
5	`record_fraud_screening` (fraud_score=0.04, recommendation=PROCEED)	FRAUD_SCREENING_COMPLETE
6	`record_compliance_check` (AML-001, passed=True)	—
7	`record_compliance_check` (KYC-001, passed=True)	COMPLIANCE_COMPLETE
8	`start_agent_session` (decision_orchestrator)	—
9	`generate_decision` (APPROVE, confidence=0.92)	PENDING_DECISION
10	`record_human_review` (override=False, final_decision=APPROVED)	APPROVED
11	Query `ledger://applications/MCP-LIFECYCLE-001/compliance` (after `daemon.run_once()`)	Complete compliance audit view returned

After step 10, the test runs `daemon.run_once()` to materialise all projection writes, then queries both resources:

```
# Application summary resource
resource_content = await client.read_resource("ledger://applications/MCP-LIFECYCLE-001
state = json.loads(resource_content[0].text)
assert state["application_id"] == "MCP-LIFECYCLE-001"
assert state["decision"] == "APPROVE"

# Compliance audit view resource (step 11)
compliance_content = await client.read_resource(
    "ledger://applications/MCP-LIFECYCLE-001/compliance"
)
compliance = json.loads(compliance_content[0].text)
assert "AML-001" in compliance["rules_passed"]   # event from step 6
```

```
assert "KYC-001" in compliance["rules_passed"]   # event from step 7
assert compliance["verdict"] == "CLEAR"
```

Step 11 confirms the compliance audit view contains a complete record of all compliance events recorded in steps 6 and 7. Both rules are present because the daemon ran `run_once()` before the resource query, atomically materialising the projection from the events written by steps 1-10.

6.3. Precondition Enforcement

Test: `test_tool_precondition_error_is_structured`

Calls `record_credit_analysis` without a prior `start_agent_session`. Asserts:

- `success == False`
- Response contains `error_type` field (typed, not a raw exception string)
- Response contains `suggested_action` field

This proves LLM-consumable error design: a consuming LLM that receives an unstructured error string cannot reason about what to do next. A typed error with `suggested_action: "call_start_agent_session_first"` enables autonomous recovery.

6.4. Compliance Temporal Query

Test: `test_compliance_resource_temporal_query`

Queries `ledger://applications/{id}/compliance?as_of={timestamp}` for a past point in time. Asserts that the compliance state as it existed at that moment is returned, distinct from the current state. This is the regulatory time-travel requirement: "What was the compliance status at time T?" is answered by the `ComplianceAuditView` projection's `get_compliance_at(application_id, timestamp)` method.

6.5. Health Resource

Test: `test_health_resource_reports_all_projection_lags`

Queries `ledger://ledger/health`. Asserts the response contains per-projection `lag_ms` values for all three projections. This is the operations watchdog endpoint: it is queried first when a projection appears stale.

6.6. What the Trace Proves About CQRS

The lifecycle trace (steps 1-11) demonstrates two structural CQRS properties simultaneously.

First, every state-changing operation in steps 1-10 flows through an MCP tool (command side), then a command handler, then `EventStore.append()`. No direct database writes occur from outside the store. The write path is: MCP tool call, command handler invocation, aggregate replay from `load_stream()`, business rule validation, append to `events` table.

Second, steps 11 and the application summary query read final state exclusively through MCP resources (query side): `ledger://applications/{id}` reads from the `application_summary` projection table, and `ledger://applications/{id}/compliance` reads from the `compliance_audit_view` projection table. Neither resource calls `load_stream()` or replays the event stream. This is confirmed directly by the `test_resource_reads_from_projection_not_stream` test, which asserts that a resource read returns projection data and completes under 50ms (the resource read SLO); stream replay at that scale would take substantially longer. The write path and the read path share only the `events` table as a common ancestor and share no code path.

---

7. Bonus Phase 6: What-If and Regulatory Package

Phase 6 is fully implemented at `what_if/projector.py` and `regulatory/package.py`. A sample regulatory package output is stored at `artifacts/regulatory_package_NARR05.json`. Documented counterfactual scenarios are at `scripts/demos/what_if_scenarios.json`.

7.1. What-If Projector (`what_if/projector.py`)

`run_what_if()` replays a real event stream but substitutes different events at a branch point, then compares projection outcomes under both the real and counterfactual paths.

Never writes to the store. All projection instances are created fresh for each scenario and discarded after the function returns.

Causal dependency filtering: After the branch event is suppressed, any downstream event whose `causation_id` traces back to a suppressed event is also suppressed. The set of suppressed event IDs grows transitively, so a multi-hop causal chain is handled correctly.

```
@dataclass
class WhatIfResult:
    application_id: str
    branch_at_event_type: str
    branch_position: int              # stream_position of the branched event
    real_outcome: dict[str, Any]
    counterfactual_outcome: dict[str, Any]
```

```
    events_before_branch: int
    counterfactual_events_injected: int
    real_events_suppressed: int       # causally dependent real events skipped
    divergence_summary: list[str]     # human-readable list of field differences
```

Documented counterfactual scenarios (`scripts/demos/what_if_scenarios.json`):

Scenario	Branch at	Counterfactual	Expected divergence
HIGH risk, same confidence	`CreditAnalysisCompleted`	`risk_tier=HIGH`	`risk_tier` , downstream `decision` suppressed
HIGH risk + confidence below 0.60 floor	`CreditAnalysisCompleted`	`risk_tier=HIGH, confidence=0.55`	`risk_tier` , `decision=None` (forced REFER by policy)
Fraud score above 0.60 threshold	`FraudScreeningCompleted`	`fraud_score=0.75`	`fraud_score` only; `DecisionGenerated` is causally independent

The third scenario demonstrates a non-obvious property of single-parent causation: `DecisionGenerated.causation_id` points to `CreditAnalysisCompleted` , not `FraudScreeningCompleted` . Branching at the fraud event does not suppress the decision event; the divergence is visible only in the read model (`fraud_score` field), not in the final decision outcome.

7.2. Regulatory Package Generator (`regulatory/package.py`)

`generate_regulatory_package(store, application_id, examination_date)` produces a self-contained JSON package containing everything a regulator needs to independently verify the decision history without querying the live database.

Package contents:

Field	Description
`event_stream`	Complete event stream up to `examination_date`, full payloads, in stream order
`projection_state_at_examination`	Application summary + compliance state derived by replaying events up to the cutoff
`audit_chain`	SHA-256 hash chain result: `integrity_hash`, `events_verified`, verification instructions
`ai_agent_provenance`	Per-agent: `model_version`, `model_deployment_id`, `input_data_hash`, `confidence_score`
`narrative`	Plain-English timeline, one sentence per significant event type, timestamped

Independent verification: The `audit_chain` section includes the hash algorithm and canonical serialisation instructions so a regulator can recompute hashes from the raw `event_stream` array without trusting the application layer:

```
"note": "Hash computed inline for this package. To verify: SHA-256 each
event payload (canonical JSON, sorted keys), concatenate the hashes in
stream_position order, then SHA-256 the concatenation."
```

Note on inline vs. stored hash: `generate_regulatory_package()` computes the hash chain inline (read-only) rather than calling `run_integrity_check()`, which writes an `AuditIntegrityCheckRun` event to the store. This preserves the guarantee that the function never writes to the store while still providing a verifiable hash.

7.3. Sample Output (`artifacts/regulatory_package_NARR05.json`)

A real regulatory package generated for application `NARR05-COMP068` (the human override scenario from NARR-05). The package was generated on 2026-03-24 and contains the full event stream from `ApplicationSubmitted` through the human reviewer overriding the orchestrator's recommendation.

Sample narrative section from that package:

```
[2026-03-24T20:31:30.637446+00:00] Application NARR05-COMP068 was submitted by
  applicant COMP-068 requesting 750000 USD.
[...] Credit analysis completed: risk tier HIGH, model v2.3.
[...] Fraud screening completed: score 0.05, risk level LOW.
[...] Compliance rule AML-001 (Anti-Money Laundering) passed.
[...] Compliance rule KYC-001 (Know Your Customer) passed.
```

```
[...] Decision generated: DECLINE with confidence 0.78.
[...] Human review by REVIEWER-007: final decision APPROVED.
[...] Application approved for 750000 USD.
```

This narrative is generated entirely from event type templates applied to stored payloads; it does not rely on any external LLM call and is fully reproducible from the `event_stream` array in the package.

---

8. Limitations and Reflection

8.1. What the Implementation Does Not Handle

Stream write authority enforcement in `EventStore.append()`. The architecture document specifies prefix validation: checking that the `agent_id` in metadata is authorised to write to the given `stream_id` prefix. For example, a compliance agent should not be able to append to `loan-{id}`. This defence-in-depth check is not implemented in the store layer. The primary enforcement is structural (agents do not hold references to other streams), but the store-level guard is absent. An agent with a bug or a compromised session could append an arbitrary event to any stream. *Acceptable in a first production deployment if the MCP server runs as a single trusted process with no external API surface. Not acceptable if agents are deployed as separate processes with independent credentials or if the MCP server is exposed to untrusted callers: in those configurations the store-level guard is the only defence against a buggy or compromised agent writing to the wrong stream.*

Projection daemon checkpoint is not atomic with projection writes in PostgreSQL. As documented in DESIGN.md Section 6, the `_save_checkpoint()` call is not inside the same database transaction as `projection.handle(event)`. For the `InMemoryEventStore` used in tests, this is safe. For a production PostgreSQL deployment where projections write to database tables, a crash between handler commit and checkpoint save causes event reprocessing on restart. All projection handlers are made idempotent to compensate, but this shifts the correctness burden to the handler rather than the infrastructure. *Not acceptable in a first production deployment where projections write to PostgreSQL tables. The correctness guarantee depends on every future handler author knowing an undocumented rule; a non-idempotent handler introduced by a handler author who is unaware of the rule will silently double-count on crash recovery, with no detection mechanism and no recovery path short of a full projection rebuild.*

Connection pool is small for high-concurrency scenarios. The `EventStore` initialises with `min_size=2, max_size=10` connections. Under 100 concurrent applications (400 concurrent

writers), pool exhaustion is the more likely bottleneck before OCC collisions become significant. A production deployment would need pool sizing analysis against the expected concurrency profile. *Acceptable in a first production deployment at low-to-medium load (under 20 concurrent writers). Pool sizing must be revisited before any load test above 50 concurrent applications, where connection queue depth becomes the dominant tail-latency contributor and the 500ms* `ApplicationSummary` *SLO may be breached.*

`DecisionGenerated` v1-to-v2 upcaster requires a store lookup. The v2 `model_versions` dict is reconstructed by loading each contributing agent session's `AgentContextLoaded` event. This means the upcaster is not a pure function: it performs I/O. The architecture document specifies that upcasters must be pure. In the current implementation, this field is set to `{}` (empty dict) as the safe default, which is honest but means historical decisions do not carry model version metadata until a store-side migration is written. *Acceptable in a first production deployment: the safe default (* `model_versions={}` *) is used, and* `model_versions` *is not used in any regulatory calculation or policy enforcement. Not acceptable once model version provenance becomes a regulatory reporting requirement; the correct fix at that point is a store-side migration that backfills the field from audit records, not a revised upcaster.*

8.2. Narrative Test Status

All five narrative scenarios now pass:

```
tests/test_narratives.py::test_narr01_concurrent_occ_collision PASSED
tests/test_narratives.py::test_narr02_document_extraction_failure PASSED
tests/test_narratives.py::test_narr03_agent_crash_recovery PASSED
tests/test_narratives.py::test_narr04_compliance_hard_block PASSED
tests/test_narratives.py::test_narr05_human_override PASSED
```

NARR-01 confirms that two concurrent `CreditAnalysisAgent` instances on the same application resolve to exactly one `CreditAnalysisCompleted` event, with the loser appending `AgentSessionFailed`.

NARR-03 confirms the full Gas Town crash recovery sequence: the crashed session stream contains `AgentSessionFailed(recoverable=True, last_successful_node=...)`, the recovery session appends `AgentSessionRecovered(context_source="prior_session_replay:{id}")`, and work resumes from the node after the last completed one.

NARR-05 confirms the human override path: the orchestrator recommends DECLINE, the loan officer overrides to APPROVE, and the final `ApplicationApproved` event carries `conditions` with the required entries.

8.3. Full Test Suite Summary

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/ --ignore=tests/test_schema.py --ignore=tests/phase1 \
                         --ignore=tests/phase2 --ignore=tests/phase3 -v

Platform: Python 3.12.3, pytest 8.4.2, asyncio mode=auto
Database: PostgreSQL 16 (Docker, port 5433)

tests/test_event_store.py          14 passed   (EventStore core + OCC + outbox)
tests/test_concurrency.py           1 passed   (double-decision OCC)
tests/test_aggregates.py           12 passed   (6 business rules, state machine)
tests/test_command_handlers.py      6 passed   (4-step handler pattern)
tests/test_projections.py           9 passed   (SLO, rebuild, idempotency, fault-tolerance
tests/test_schema_and_generator.py 10 passed   (event registry, simulator, seed data)
tests/test_upcasting.py             3 passed   (v1→v2, immutability, null policy)
tests/test_integrity.py             4 passed   (hash chain, tamper detection, Gas Town)
tests/test_gas_town.py              2 passed   (crash recovery, NEEDS_RECONCILIATION)
tests/test_mcp_integration.py       5 passed   (lifecycle, preconditions, resources, tempo
tests/test_mcp_lifecycle.py         1 passed   (full lifecycle via MCP tools only)
tests/test_mcp_pg.py                3 passed   (PostgreSQL OCC, lifecycle, append-only)
tests/test_narratives.py            5 passed   (NARR-01 through NARR-05)
------------------------------------------------------------------------------
Total                              80 passed   22.54s

test_schema.py (database DDL verification) run separately; passes on live DB.
```

8.4. What I Would Change with More Time

The single highest-priority change is atomising the checkpoint in `_save_checkpoint()`. The current design relies on every projection handler being idempotent as a correctness guarantee. This is not documented as a requirement of the `Projection` interface, which means a future handler author can introduce a non-idempotent handler that silently double-counts under crash conditions. Wrapping both the handler call and the checkpoint update in a single PostgreSQL transaction would make the guarantee structural and remove the implicit dependency on handler authors knowing the rule.

The second priority is extracting `CreditAnalysisCompleted` and `FraudScreeningCompleted` to `credit-{id}` and `fraud-{id}` streams respectively. The current implementation collapses these onto `loan-{id}`, which is safe under a sequential pipeline but creates latent OCC risk if the pipeline is ever made concurrent. The architecture document already describes the six-stream model; aligning the implementation with it would remove a documented known gap and make concurrent multi-agent execution safe without further changes.

The third priority is adding stream write authority enforcement in `EventStore.append()`. The architecture specifies prefix validation: a compliance agent should not be able to append to `loan-{id}`. The current enforcement is structural only (agents do not hold references to other streams). A store-level guard would make the boundary explicit and catch bugs or compromised sessions that bypass the structural constraint.